Predicting LEGO Set Themes from Part Composition

AI Capstone Project #1 | NYCU Spring 2026 | 312554027 Chin-Yang Lin

Dataset: https://github.com/linjohnss/lego-theme-classification

1. Introduction

In machine learning, dataset quality often matters more than algorithm choice. This project explores dataset construction through a creative classification task: predicting which LEGO product theme a set belongs to based solely on its part composition. LEGO themes have distinctive characteristics — Star Wars uses grey palettes with many minifigures, Technic uses beams and gears instead of bricks, and Duplo uses entirely different large-format parts. We hypothesize these differences enable accurate classification from parts lists alone.

Our research questions: (1) Can ML models reliably predict LEGO themes from part-level features? (2) Which feature groups contribute most? (3) How do class balance, data volume, and dimensionality reduction affect performance?

2. Dataset

2.1 Source and Construction

We constructed our dataset from the Rebrickable open database (rebrickable.com/downloads/), which provides comprehensive LEGO data as CSV files. We downloaded 9 tables (~1.6M records) using Python's requests library. While the raw data is public, our dataset is self-constructed: we designed the feature pipeline, selected themes, and computed all features from scratch — analogous to using government weather APIs as a source for a custom prediction dataset.

2.2 Data Cleanup and Processing

The raw data required substantial cleanup. Of the 26,462 sets in the database, 6,983 (26.4%) had zero parts — these are non-buildable merchandise (keychains, bags, stationery, houseware) that must be filtered out. We also found 45,042 inventory records for 26,462 sets, meaning some sets have multiple inventory versions; we retained only version 1 (the primary inventory) to avoid duplicates. Within inventory_parts, we excluded spare parts (is_spare=True) since they are not part of the intended build. Missing values were minimal: only 7,170 missing image URLs (irrelevant for our features) and 2 colors without year data.

For theme mapping, we resolved 492 sub-themes to root themes using the hierarchical parent_id structure. We merged "Town" (1978-2004) into "City" (2005-present) as they represent the same product line. We selected 9 themes with >150 samples each, excluding non-buildable categories (Gear, Books, Collectible Minifigures) and themes too small for reliable cross-validation. Final dataset: 6,871 sets across 9 classes.

2.3 Feature Engineering

For each set, we joined 7 tables and computed 256 features in 6 groups:

Feature Group	Dims	Description
Color distribution	166	Proportion of each color (quantity-weighted)
Part category dist.	70	Proportion of each part type (bricks, plates, etc.)

Material dist.	6	Proportion by material (Plastic, Rubber, etc.)
Color statistics	7	Weighted mean/std RGB, transparency ratio
Scalar features	5	num_parts, year, unique parts/colors/categories
Minifig features	2	Minifigure count and unique minifigure count

Table 1: Feature groups (256 total dimensions).

Color and part category distributions are quantity-weighted proportions summing to 1, capturing the compositional profile rather than absolute size. Only non-spare parts from the primary inventory (version 1) are included. Color statistics (avg/std RGB) provide a compact summary of the color palette; for instance, Star Wars sets have high grey values while Friends sets have high pink/purple values.

2.4 Dataset Examples

Table 2 shows feature highlights for representative sets from different themes, illustrating how part composition reflects theme identity.

Set	Theme	Parts	Minifigs	Top Color	Top Category
75192-1	Star Wars	7541	8	L.Bl.Gray (38%)	Plates (31%)
42100-1	Technic	2573	0	D.Bl.Gray (26%)	Technic Pins (22%)
10698-1	Creator	790	0	Yellow (12%)	Bricks (35%)
10914-1	Duplo	85	3	Red (18%)	Duplo (89%)
41395-1	Friends	778	4	White (16%)	Plates (26%)
71741-1	Ninjago	5685	13	D.Bl.Gray (21%)	Plates (27%)

Table 2: Example sets showing distinctive composition per theme.

2.5 Class Distribution

The final dataset contains 6,871 LEGO sets across 9 themes. The distribution is imbalanced: City has 1,517 samples while Harry Potter has only 190 (8:1 ratio), providing a natural setting for our class balance experiments.

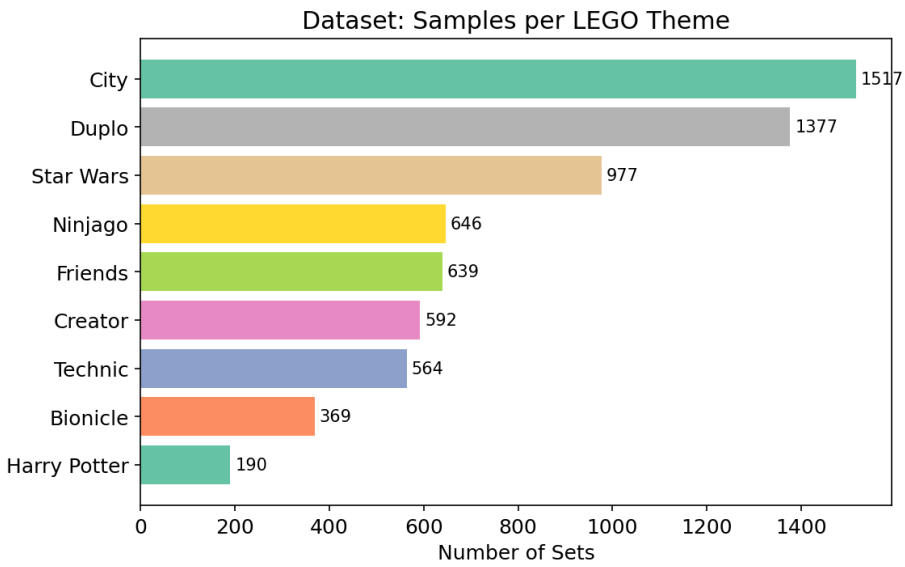

Figure 1: Samples per theme in the dataset.

3. Methods

3.1 Models

We evaluate three supervised learning models, of which at most one is deep learning-based:

Random Forest (RF): An ensemble of 300 decision trees with bootstrap aggregation and balanced class weights. RF handles high-dimensional sparse features well and provides interpretable feature importance scores. (scikit-learn RandomForestClassifier [1])

Histogram Gradient Boosting (HGB): A gradient boosting method using histogram-based binning for efficient training. Configured with 300 iterations and max_depth=8. HGB typically achieves state-of-the-art results on tabular data. (scikit-learn HistGradientBoostingClassifier [1])

Multi-Layer Perceptron (MLP): A neural network with three hidden layers (256, 128, 64 neurons), ReLU activation, and early stopping. This is our only deep learning model. Features are standardized (zero mean, unit variance) before training. (scikit-learn MLPClassifier [1])

3.2 Evaluation and Additional Methods

All models are evaluated with 5-fold Stratified Cross-Validation, preserving class proportions in each fold. We report Accuracy and Macro-averaged Precision, Recall, and F1. Features are standardized within each fold to prevent data leakage. For experiments, we also use: PCA for dimensionality reduction [1], SMOTE for minority class oversampling [2,5], and t-SNE for 2D visualization [1].

4. Experiments and Results

4.1 Baseline Performance

Model	Accuracy	Prec	Rec	F1 (Macro)
Random Forest	0.9383 +/- 0.0096	0.9339	0.9039	0.9145 +/- 0.0109
Hist Gradient Boosting	0.9550 +/- 0.0045	0.9505	0.9318	0.9392 +/- 0.0042
MLP	0.9237 +/- 0.0026	0.9023	0.8900	0.8951 +/- 0.0022

Table 2: Baseline performance (5-fold CV).

HGB achieves the best performance (95.5% accuracy, 0.939 F1), followed by RF (93.8%) and MLP (92.4%). Tree-based models outperforming MLP is consistent with literature on tabular data [3].

4.2 Per-Class Analysis

Theme	Precision	Recall	F1
Bionicle	0.984	0.981	0.982
City	0.961	0.962	0.961
Creator	0.899	0.887	0.893
Duplo	1.000	0.999	0.999
Friends	0.960	0.977	0.968
Harry Potter	0.926	0.721	0.811
Ninjago	0.927	0.929	0.928
Star Wars	0.908	0.944	0.926
Technic	0.988	0.988	0.988

Table 3: Per-class metrics (HGB, best model).

Duplo (F1=0.999) and Technic (0.988) are near-perfectly classified due to unique part systems. Harry Potter has lowest F1 (0.811) due to small sample size and overlap with other themes.

Creator (0.893) is most confused with City and Star Wars, as it uses generic bricks.

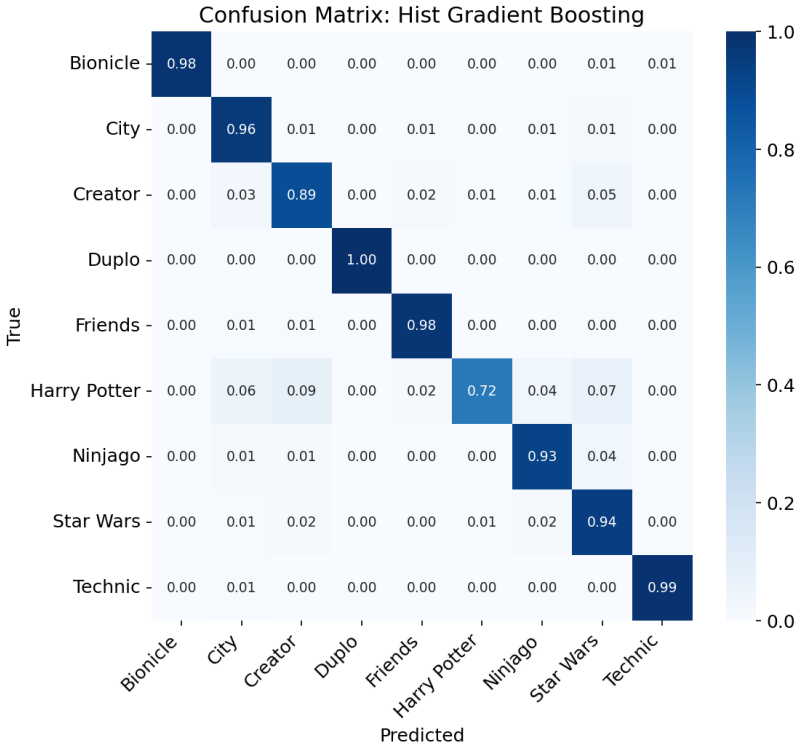

Figure 2: Normalized confusion matrix (HGB).

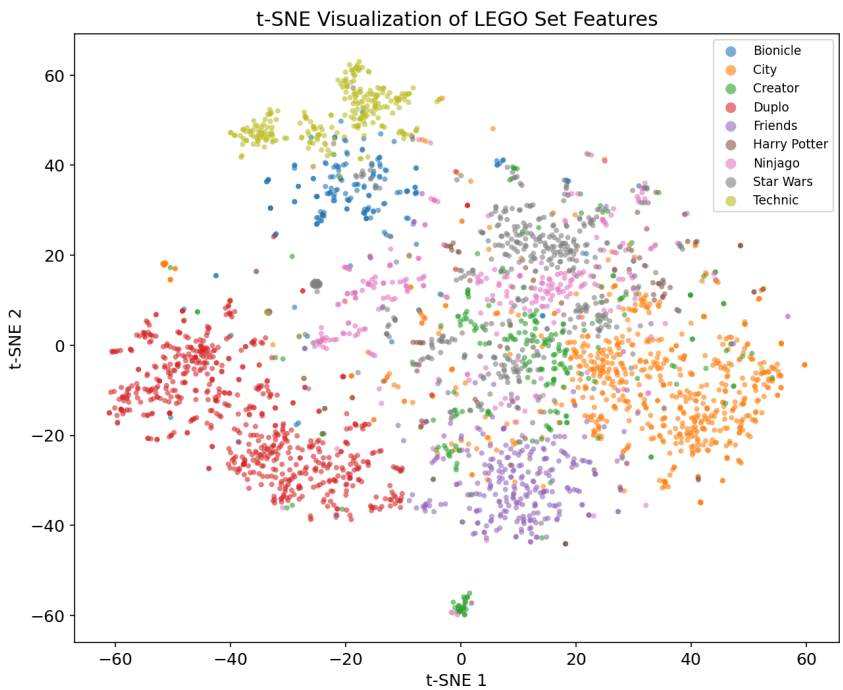

Figure 3: t-SNE visualization. Duplo (red), Technic (yellow-green), and Bionicle (blue) form distinct clusters; City/Creator/Star Wars overlap in the center.

4.3 Exp 1: Training Data Amount

We subsample training data from 20% to 100% within each CV fold.

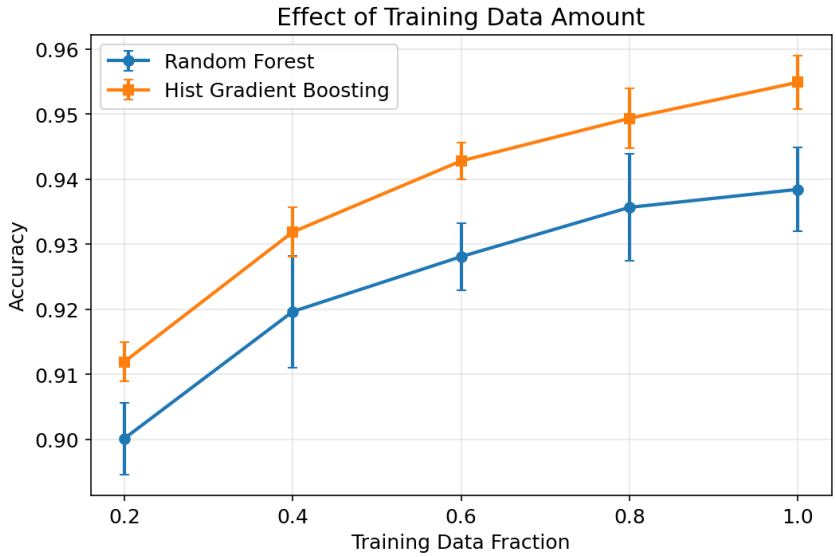

Figure 4: Accuracy vs. training data fraction.

Both models exceed 90% accuracy even at 20% data (~1,100 samples), demonstrating highly informative features. HGB consistently outperforms RF. Diminishing returns suggest the dataset size is sufficient.

4.4 Exp 2: Class Balance

We compare: (a) no balancing, (b) class_weight="balanced", (c) SMOTE oversampling.

Model	Strategy	Accuracy	F1
Random Forest	Original	0.9360	0.9110
Hist Gradient Boosting	Original	0.9550	0.9392
Random Forest	Balanced (class_weight)	0.9383	0.9145
Hist Gradient Boosting	Balanced (class_weight)	0.9543	0.9390
Random Forest	SMOTE	0.9397	0.9207
Hist Gradient Boosting	SMOTE	0.9491	0.9328

Table 4: Class balance strategies.

SMOTE improves RF F1 (0.911->0.921) by boosting Harry Potter recall, but slightly hurts HGB (0.939->0.933). Key insight: optimal balancing strategy is model-dependent.

4.5 Exp 3: PCA Dimensionality Reduction

Components	Accuracy
Full	0.9383
10	0.8492
30	0.8839
50	0.8946
100	0.8927
150	0.8932

Table 5: RF accuracy with PCA.

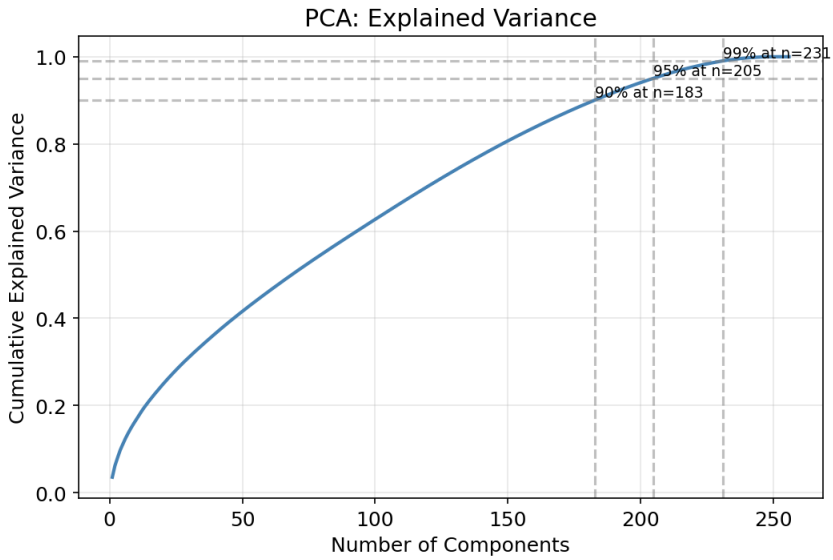

Figure 5: Cumulative explained variance. 90% at 183 components, 95% at 205.

PCA consistently hurts performance (best: 89.5% at 50 components vs 93.8% full). RF natively handles sparse high-dimensional features via feature subsampling at each split, making PCA redundant. The lost 10% variance contains discriminative signals.

4.6 Exp 4: Feature Ablation

Feature Group	#	Accuracy	F1
Color features	173	0.8213	0.7788
Part category features	70	0.8801	0.8376
Metadata features	13	0.8077	0.7481
All features	256	0.9383	0.9145

Table 6: Feature ablation (RF).

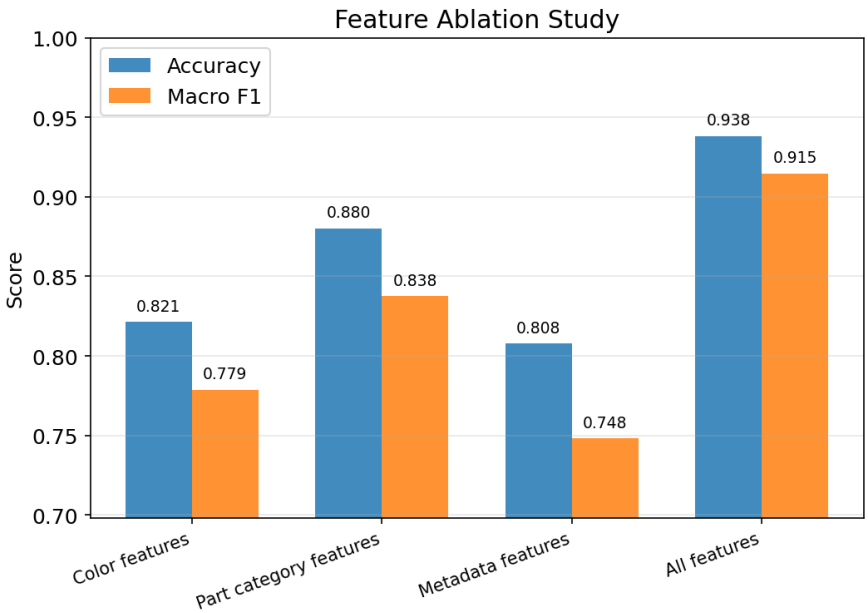

Figure 6: Accuracy and Macro F1 for each feature subset.

Part categories are the single most informative group (88.0%), which makes sense: themes like Duplo, Technic, and Bionicle use fundamentally different part types. Color features achieve 82.1%, confirming that color palettes carry theme-specific signals (Friends uses pastels, Star

Wars uses greys). Metadata alone reaches 80.8% with only 13 features — surprisingly high, largely driven by the year feature (themes are era-specific). Combining all groups yields 93.8%, demonstrating strong complementarity.

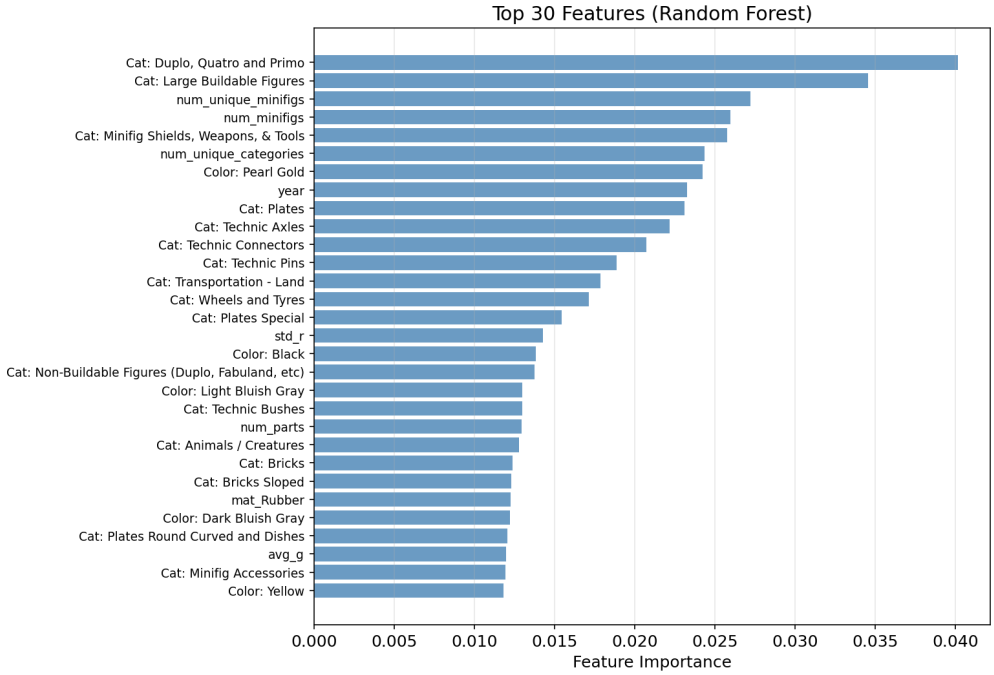

Figure 7: Top 30 features by RF importance.

The most important feature is the "Duplo, Quatro and Primo" part category, which perfectly identifies Duplo sets. Other top features include "Large Buildable Figures" (Bionicle), minifigure counts, Pearl Gold color, and the number of unique part categories.

4.7 Exp 5: Hyperparameter Sensitivity

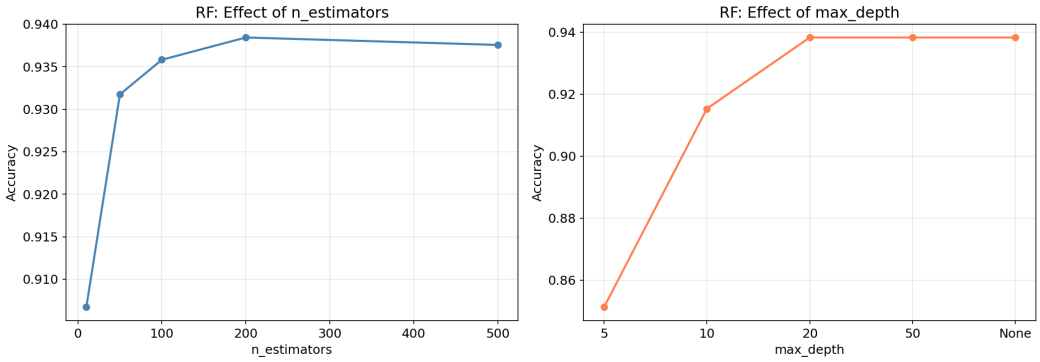

Figure 8: RF accuracy vs n_estimators (left) and max_depth (right).

Accuracy saturates at ~200 trees (93.8%) and max_depth=20. Shallow trees (depth=5) significantly underperform (85.1%), indicating moderate decision boundary complexity.

4.8 Exp 6: Data Augmentation

We apply data augmentation by duplicating training samples with added Gaussian noise (std = 0.01, 0.05, 0.1 on standardized features), effectively doubling the training set.

Model	Augmentation	Accuracy	F1
Random Forest	No augmentation	0.9384	0.9140
Hist Gradient Boosting	No augmentation	0.9549	0.9397
Random Forest	noise=0.01	0.9389	0.9158

Hist Gradient Boosting	noise=0.01	0.9478	0.9297
Random Forest	noise=0.05	0.9383	0.9141
Hist Gradient Boosting	noise=0.05	0.9482	0.9325
Random Forest	noise=0.1	0.9374	0.9156
Hist Gradient Boosting	noise=0.1	0.9478	0.9305

Table 7: Effect of data augmentation.

Augmentation provides negligible benefit for RF and slightly hurts HGB. This makes sense: our features are compositional proportions (summing to 1) and adding Gaussian noise breaks this structure. Unlike image data, where augmentation simulates realistic variations, adding noise to part distributions creates unrealistic compositions. This suggests that augmentation strategies must respect the feature semantics — a key lesson for tabular data.

4.9 Exp 7: Year Feature Importance

The year feature ranked highly in feature importance. To test whether models learn actual compositional patterns or simply memorize temporal associations, we remove year and re-evaluate.

Model	Condition	Accuracy	F1
Random Forest	With year	0.9384	0.9140
Random Forest	Without year	0.9317	0.9070
Hist Gradient Boosting	With year	0.9549	0.9397
Hist Gradient Boosting	Without year	0.9480	0.9298
MLP	With year	0.9237	0.8951
MLP	Without year	0.9229	0.8979

Table 8: Effect of removing the year feature.

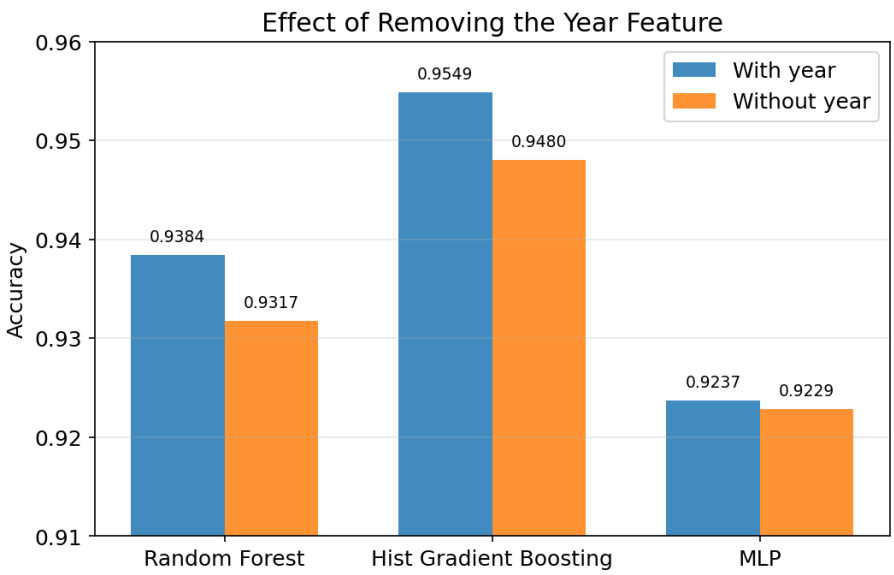

Figure 9: Accuracy with and without the year feature.

Removing year reduces RF accuracy by 0.7% and HGB by 0.7%, confirming that year provides useful but not essential information. Interestingly, MLP is barely affected (0.08% drop), suggesting it relies less on this single feature. The models still achieve >93% accuracy without year, proving they learn genuine compositional patterns rather than simply memorizing which themes existed in which era.

5. Discussion

5.1 Expected vs. Observed Results

Our results largely confirm the initial hypotheses. Themes with unique part systems — Duplo (large-format parts), Technic (beams, pins, gears), and Bionicle (constraction figures) — are trivially classified with F1 scores above 0.98. This validates our approach of using part category distributions as features, since these themes occupy entirely different regions of the part category space.

Among standard-brick themes, Friends (F1=0.968) is easier to classify than expected, likely because its distinctive pastel color palette (lavender, medium azure, bright pink) provides a strong signal. Creator (F1=0.893) is hardest, as expected, since it uses generic bricks that overlap with City and Star Wars. Tree-based models outperform MLP, consistent with recent benchmarks showing gradient boosting dominates neural networks on structured tabular data [3].

One surprising finding: PCA dimensionality reduction consistently hurts performance. We expected some compression to improve results by removing noise, but our sparse feature vectors (most sets use fewer than 20 of 166 possible colors) contain crucial discriminative signals in rare-but-informative color dimensions that PCA discards.

5.2 Factors Affecting Results

Several dataset characteristics significantly influence our results:

(1) Class imbalance: Harry Potter's small sample size (190 sets, 2.8% of data) results in the lowest recall across all models. SMOTE partially mitigates this for Random Forest (F1: 0.911 to 0.921) but not for HGB, suggesting that HGB's gradient-based learning already handles imbalance more gracefully.

(2) Feature sparsity: Most sets use fewer than 20 of the 166 possible colors, creating very sparse feature vectors. Tree-based models handle this naturally (each split examines one feature), while MLP may struggle with the curse of dimensionality.

(3) Temporal effects: The year feature captures theme-specific temporal patterns. Bionicle sets are concentrated in 2001-2016, while Ninjago started in 2011. The merged Town/City class spans 1978-2026, potentially introducing within-class variance due to evolving part design over decades.

5.3 Future Work

Given more time, several extensions would be valuable: (1) Using set images from Rebrickable for CNN-based visual classification, comparing part-composition features vs. visual features. (2) Incorporating BrickLink secondary market price data for a regression task (predicting set value from composition). (3) Applying graph neural networks where parts are nodes and co-occurrence relationships are edges, potentially capturing spatial patterns. (4) Expanding to all 50+ themes with hierarchical classification. (5) Studying temporal drift: whether models trained on older sets generalize to newer ones.

5.4 Lessons Learned and Remaining Questions

This project reinforced key insights about data-centric ML: (1) Feature engineering matters more than model selection — our carefully designed features achieve 90%+ accuracy even with 20% of the data using a simple Random Forest. (2) Domain knowledge drives good

features — understanding that Duplo uses different parts, that Star Wars has a grey palette, and that Technic uses beams guided our design. (3) Standard techniques (PCA, SMOTE) can hurt when their assumptions don't match the data structure. Understanding when NOT to apply a technique is as important as knowing how.

Remaining questions: (1) Why does PCA consistently degrade performance — is this specifically due to the sparsity structure of our features, or would any linear dimensionality reduction method (e.g., LDA) fail? Would non-linear methods like autoencoders preserve the discriminative signals? (2) Can our models generalize across time — i.e., if trained only on pre-2015 sets, would they accurately classify 2020+ sets where part designs have evolved? (3) Creator sets are hardest to classify because they use generic parts by design — would adding part co-occurrence features (which specific parts appear together) improve Creator classification without harming other classes?

6. References

[1] Pedregosa et al. "Scikit-learn: ML in Python." JMLR 12 (2011): 2825-2830.

[2] Lemaitre et al. "Imbalanced-learn: A Python Toolbox." JMLR 18(17) (2017): 1-5.

[3] Grinsztajn et al. "Why do tree-based models still outperform deep learning on tabular data?" NeurIPS 2022.

[4] Rebrickable.com. "LEGO Database." https://rebrickable.com/downloads/

[5] Chawla et al. "SMOTE: Synthetic Minority Over-sampling." JAIR 16 (2002): 321-357.

Appendix: Program Code

All code written in Python 3 using scikit-learn, pandas, numpy, matplotlib.

feature_engineering.py

Build dataset from raw CSV

```
"""
Feature Engineering: Build LEGO theme classification dataset from Rebrickable raw data.

Reads raw CSV files from data/, joins tables, computes per-set features
(color distribution, part category distribution, material distribution,
color statistics, minifig counts), and outputs a single dataset.csv.
"""

import os
import numpy as np
import pandas as pd

DATA_DIR = os.path.join(os.path.dirname(__file__), 'data')
OUTPUT_DIR = os.path.join(os.path.dirname(__file__), 'output')

# 9 target themes (Town merged into City)
TARGET_THEMES = [
    'Star Wars', 'Friends', 'Ninjago', 'Technic',
    'Bionicle', 'City', 'Creator', 'Duplo', 'Harry Potter'
]

# Root theme names that should be merged into City
CITY_MERGE = {'City', 'Town'}


def load_csv(name):
    return pd.read_csv(os.path.join(DATA_DIR, f'{name}.csv'))


def build_theme_map(themes_df):
    """Map every theme_id to its root theme name."""
    theme_dict = themes_df.set_index('id')[['name', 'parent_id']].to_dict('index')
    cache = {}

    def get_root(tid):
        if tid in cache:
            return cache[tid]
        if tid not in theme_dict:
            return None
        info = theme_dict[tid]
        pid = info['parent_id']
        if pd.isna(pid):
            cache[tid] = info['name']
        else:
            cache[tid] = get_root(int(pid))
        return cache[tid]

    for tid in theme_dict:
        get_root(tid)
    return cache


def hex_to_rgb(hex_str):
    """Convert hex color string to (R, G, B) tuple."""
    try:
        h = hex_str.lstrip('#')
        return int(h[0:2], 16), int(h[2:4], 16), int(h[4:6], 16)
    except (ValueError, TypeError, AttributeError):
        return (128, 128, 128)


def run():
    """Main pipeline: load raw CSVs -> filter -> join -> compute features -> save dataset."""
    print("=== Feature Engineering ===")

    # --- Load all 9 raw CSV tables from Rebrickable ---
    themes = load_csv('themes')       # 492 themes with hierarchical parent_id
    sets = load_csv('sets')           # 26,462 sets with year, theme_id, num_parts
    inventories = load_csv('inventories')  # Maps sets to inventory IDs (may have multiple versions)
    inv_parts = load_csv('inventory_parts')    # 1.49M rows: detailed parts per inventory
    inv_minifigs = load_csv('inventory_minifigs')  # Minifigures per inventory
    colors = load_csv('colors')       # 275 colors with RGB hex values and transparency flag
    parts = load_csv('parts')         # 61K parts with category and material
    part_cats = load_csv('part_categories')  # 76 part categories (Bricks, Plates, Technic, etc.)

    # --- Step 1: Map themes and filter sets ---
    theme_map = build_theme_map(themes)
    sets['root_theme'] = sets['theme_id'].map(theme_map)

    # Merge Town into City
    sets['root_theme'] = sets['root_theme'].replace('Town', 'City')
```

```
# Filter: num_parts > 0 and target themes only
sets_filtered = sets[(sets['num_parts'] > 0) & (sets['root_theme'].isin(TARGET_THEMES))].copy()
print(f"Sets after filtering: {len(sets_filtered)}")
print(f"Class distribution:\n{sets_filtered['root_theme'].value_counts().to_string()}\n")

# --- Step 2: Join to inventories (version 1 only) ---
inv_v1 = inventories[inventories['version'] == 1][['id', 'set_num']]
sets_inv = sets_filtered.merge(inv_v1, on='set_num', how='inner')
sets_inv = sets_inv.rename(columns={'id': 'inventory_id'})
print(f"Sets with inventory: {len(sets_inv)}")

# --- Step 3: Compute part-based features ---
# Filter spare parts
inv_parts_used = inv_parts[inv_parts['is_spare'] == False][['inventory_id', 'part_num', 'color_id', 'quantity']]

# Keep only inventories in our set
valid_inv_ids = set(sets_inv['inventory_id'])
inv_parts_used = inv_parts_used[inv_parts_used['inventory_id'].isin(valid_inv_ids)]
print(f"Inventory part rows for our sets: {len(inv_parts_used)}")

# Join part info (category, material)
parts_info = parts[['part_num', 'part_cat_id', 'part_material']]
inv_parts_enriched = inv_parts_used.merge(parts_info, on='part_num', how='left')

# Join color info (rgb, is_trans)
colors_info = colors[['id', 'rgb', 'is_trans']].rename(columns={'id': 'color_id'})
inv_parts_enriched = inv_parts_enriched.merge(colors_info, on='color_id', how='left')

# Parse RGB values
rgb_values = inv_parts_enriched['rgb'].apply(hex_to_rgb)
inv_parts_enriched['r'] = rgb_values.apply(lambda x: x[0])
inv_parts_enriched['g'] = rgb_values.apply(lambda x: x[1])
inv_parts_enriched['b'] = rgb_values.apply(lambda x: x[2])

# --- 3a. Color distribution (275 dims) ---
print("Computing color distribution...")
color_pivot = inv_parts_enriched.groupby(['inventory_id', 'color_id'])['quantity'].sum().unstack(fill_value=0)
# Normalize each row to proportions
color_totals = color_pivot.sum(axis=1)
color_dist = color_pivot.div(color_totals, axis=0)
color_dist.columns = [f'color_{c}' for c in color_dist.columns]

# --- 3b. Part category distribution (76 dims) ---
print("Computing part category distribution...")
cat_pivot = inv_parts_enriched.groupby(['inventory_id', 'part_cat_id'])['quantity'].sum().unstack(fill_value=0)
cat_totals = cat_pivot.sum(axis=1)
cat_dist = cat_pivot.div(cat_totals, axis=0)
cat_dist.columns = [f'cat_{c}' for c in cat_dist.columns]

# --- 3c. Material distribution ---
print("Computing material distribution...")
mat_pivot = inv_parts_enriched.groupby(['inventory_id', 'part_material'])['quantity'].sum().unstack(fill_value=0)
mat_totals = mat_pivot.sum(axis=1)
mat_dist = mat_pivot.div(mat_totals, axis=0)
mat_dist.columns = [f'mat_{c}' for c in mat_dist.columns]

# --- 3d. Color statistics ---
print("Computing color statistics...")
def weighted_color_stats(group):
    q = group['quantity'].values.astype(float)
    total_q = q.sum()
    if total_q == 0:
        return pd.Series({'avg_r': 128, 'avg_g': 128, 'avg_b': 128,
                          'std_r': 0, 'std_g': 0, 'std_b': 0, 'prop_transparent': 0})
    weights = q / total_q
    r, g, b = group['r'].values, group['g'].values, group['b'].values
    avg_r = np.average(r, weights=weights)
    avg_g = np.average(g, weights=weights)
    avg_b = np.average(b, weights=weights)
    std_r = np.sqrt(np.average((r - avg_r) ** 2, weights=weights))
    std_g = np.sqrt(np.average((g - avg_g) ** 2, weights=weights))
    std_b = np.sqrt(np.average((b - avg_b) ** 2, weights=weights))
    trans = group['is_trans'].values.astype(float)
    prop_trans = np.average(trans, weights=weights)
    return pd.Series({'avg_r': avg_r, 'avg_g': avg_g, 'avg_b': avg_b,
                      'std_r': std_r, 'std_g': std_g, 'std_b': std_b,
                      'prop_transparent': prop_trans})

color_stats = inv_parts_enriched.groupby('inventory_id').apply(weighted_color_stats, include_groups=False)

# --- 3e. Scalar features ---
print("Computing scalar features...")
scalar_feats = inv_parts_enriched.groupby('inventory_id').agg(
    num_unique_parts=('part_num', 'nunique'),
    num_unique_colors=('color_id', 'nunique'),
    num_unique_categories=('part_cat_id', 'nunique'),
)

# --- Step 4: Minifig features ---
print("Computing minifig features...")
inv_minifigs_filtered = inv_minifigs[inv_minifigs['inventory_id'].isin(valid_inv_ids)]
minifig_feats = inv_minifigs_filtered.groupby('inventory_id').agg(
    num_minifigs=('quantity', 'sum'),
    num_unique_minifigs=('fig_num', 'nunique'),
)
```

```
    # --- Step 5: Assemble final dataset ---
    print("Assembling final dataset...")
    # Start with set info
    dataset = sets_inv[['set_num', 'inventory_id', 'root_theme', 'num_parts', 'year']].copy()
    dataset = dataset.set_index('inventory_id')

    # Join all feature groups
    dataset = dataset.join(color_dist, how='left')
    dataset = dataset.join(cat_dist, how='left')
    dataset = dataset.join(mat_dist, how='left')
    dataset = dataset.join(color_stats, how='left')
    dataset = dataset.join(scalar_feats, how='left')
    dataset = dataset.join(minifig_feats, how='left')

    # Fill NaN (sets with no minifigs get 0)
    dataset['num_minifigs'] = dataset['num_minifigs'].fillna(0).astype(int)
    dataset['num_unique_minifigs'] = dataset['num_unique_minifigs'].fillna(0).astype(int)
    dataset = dataset.fillna(0)

    # Rename label column
    dataset = dataset.rename(columns={'root_theme': 'theme'})
    dataset = dataset.reset_index(drop=True)

    # --- Save ---
    os.makedirs(OUTPUT_DIR, exist_ok=True)
    output_path = os.path.join(OUTPUT_DIR, 'dataset.csv')
    dataset.to_csv(output_path, index=False)

    # Summary
    feature_cols = [c for c in dataset.columns if c not in ['set_num', 'theme']]
    summary = (
        f"Dataset shape: {dataset.shape}\n"
        f"Feature count: {len(feature_cols)}\n"
        f"Samples per class:\n{dataset['theme'].value_counts().to_string()}\n\n"
        f"Feature groups:\n"
        f"  Color distribution: {sum(1 for c in feature_cols if c.startswith('color_'))}\n"
        f"  Part category distribution: {sum(1 for c in feature_cols if c.startswith('cat_'))}\n"
        f"  Material distribution: {sum(1 for c in feature_cols if c.startswith('mat_'))}\n"
        f"  Color statistics: 7 (avg/std RGB + prop_transparent)\n"
        f"  Scalar features: 5 (num_parts, year, num_unique_parts/colors/categories)\n"
        f"  Minifig features: 2 (num_minifigs, num_unique_minifigs)\n"
    )
    print(summary)

    info_path = os.path.join(OUTPUT_DIR, 'dataset_info.txt')
    with open(info_path, 'w') as f:
        f.write(summary)

    print(f"Dataset saved to {output_path}")
    print(f"Summary saved to {info_path}")
    return dataset


if __name__ == '__main__':
    run()
```

train_evaluate.py

Model training and CV evaluation

```
"""
Model Training and Evaluation: Baseline performance with 5-fold Stratified CV.

Models: Random Forest, Histogram Gradient Boosting, MLP (neural network).
Metrics: Accuracy, Macro Precision/Recall/F1, Confusion Matrix.
Results saved to output/ as JSON for use by visualize.py.
"""

import os
import json
import numpy as np
import pandas as pd
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix)

OUTPUT_DIR = os.path.join(os.path.dirname(__file__), 'output')


def get_models():
    """Return dict of model name -> model instance."""
    return {
        'Random Forest': RandomForestClassifier(
            n_estimators=300, max_depth=None, class_weight='balanced',
            random_state=42, n_jobs=-1
        ),
        'Hist Gradient Boosting': HistGradientBoostingClassifier(
            max_iter=300, max_depth=8, learning_rate=0.1, random_state=42
```

```
        ),
        'MLP': MLPClassifier(
            hidden_layer_sizes=(256, 128, 64), activation='relu',
            max_iter=500, early_stopping=True, validation_fraction=0.15,
            random_state=42
        ),
    }


def load_dataset():
    """Load dataset and return X, y, feature_names, label_encoder."""
    df = pd.read_csv(os.path.join(OUTPUT_DIR, 'dataset.csv'))
    feature_cols = [c for c in df.columns if c not in ['set_num', 'theme']]
    X = df[feature_cols].values
    le = LabelEncoder()
    y = le.fit_transform(df['theme'])
    return X, y, feature_cols, le


def run():
    print("=== Model Training & Evaluation ===")
    X, y, feature_names, le = load_dataset()
    print(f"Dataset: {X.shape[0]} samples, {X.shape[1]} features, {len(le.classes_)} classes")
    print(f"Classes: {list(le.classes_)}\n")

    models = get_models()
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

    all_results = {}

    for model_name, model_template in models.items():
        print(f"--- {model_name} ---")
        fold_metrics = []
        fold_cms = []

        for fold_idx, (train_idx, test_idx) in enumerate(skf.split(X, y)):
            X_train, X_test = X[train_idx], X[test_idx]
            y_train, y_test = y[train_idx], y[test_idx]

            # Scale features
            scaler = StandardScaler()
            X_train_s = scaler.fit_transform(X_train)
            X_test_s = scaler.transform(X_test)

            # Clone model (fresh instance each fold)
            from sklearn.base import clone
            model = clone(model_template)
            model.fit(X_train_s, y_train)
            y_pred = model.predict(X_test_s)

            acc = accuracy_score(y_test, y_pred)
            prec = precision_score(y_test, y_pred, average='macro', zero_division=0)
            rec = recall_score(y_test, y_pred, average='macro', zero_division=0)
            f1 = f1_score(y_test, y_pred, average='macro', zero_division=0)
            cm = confusion_matrix(y_test, y_pred, labels=np.arange(len(le.classes_)))

            fold_metrics.append({'accuracy': acc, 'precision': prec, 'recall': rec, 'f1': f1})
            fold_cms.append(cm)
            print(f"  Fold {fold_idx+1}: Acc={acc:.4f}, F1={f1:.4f}")

        # Aggregate
        metrics_df = pd.DataFrame(fold_metrics)
        mean_metrics = metrics_df.mean().to_dict()
        std_metrics = metrics_df.std().to_dict()
        avg_cm = np.mean(fold_cms, axis=0)

        all_results[model_name] = {
            'mean': mean_metrics,
            'std': std_metrics,
            'confusion_matrix': avg_cm.tolist(),
            'fold_metrics': fold_metrics,
        }

        print(f"  Mean: Acc={mean_metrics['accuracy']:.4f}±{std_metrics['accuracy']:.4f}, "
              f"F1={mean_metrics['f1']:.4f}±{std_metrics['f1']:.4f}\n")

    # Save results
    results_path = os.path.join(OUTPUT_DIR, 'baseline_results.json')
    with open(results_path, 'w') as f:
        json.dump(all_results, f, indent=2)

    # Save class names
    classes_path = os.path.join(OUTPUT_DIR, 'class_names.json')
    with open(classes_path, 'w') as f:
        json.dump(list(le.classes_), f)

    # Save feature names
    features_path = os.path.join(OUTPUT_DIR, 'feature_names.json')
    with open(features_path, 'w') as f:
        json.dump(feature_names, f)

    # Train final RF on full data for feature importance
    print("Training final Random Forest for feature importance...")
    scaler = StandardScaler()
    X_s = scaler.fit_transform(X)
```

```
    rf_final = RandomForestClassifier(
        n_estimators=300, max_depth=None, class_weight='balanced',
        random_state=42, n_jobs=-1
    )
    rf_final.fit(X_s, y)
    importance = rf_final.feature_importances_
    importance_path = os.path.join(OUTPUT_DIR, 'feature_importance.json')
    with open(importance_path, 'w') as f:
        json.dump(dict(zip(feature_names, importance.tolist())), f)

    # Print summary table
    print("\n=== Summary ===")
    print(f"{'Model':<25} {'Accuracy':>12} {'Precision':>12} {'Recall':>12} {'F1':>12}")
    print("-" * 73)
    for name, res in all_results.items():
        m, s = res['mean'], res['std']
        print(f"{name:<25} {m['accuracy']:.4f}±{s['accuracy']:.4f} "
              f"{m['precision']:.4f}±{s['precision']:.4f} "
              f"{m['recall']:.4f}±{s['recall']:.4f} "
              f"{m['f1']:.4f}±{s['f1']:.4f}")

    print(f"\nResults saved to {results_path}")
    return all_results


if __name__ == '__main__':
    run()
```

experiments.py

All 5 experiments

```
"""
Experiments: Investigate how dataset characteristics affect model performance.

Experiment 1: Effect of training data amount
Experiment 2: Class balance strategies (original / balanced / SMOTE)
Experiment 3: PCA dimensionality reduction
Experiment 4: Feature ablation (color-only / category-only / metadata-only / all)
Experiment 5: Hyperparameter sensitivity (Random Forest)
"""

import os
import json
import numpy as np
import pandas as pd
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.decomposition import PCA
from sklearn.metrics import accuracy_score, f1_score
from sklearn.base import clone
from sklearn.utils import resample

OUTPUT_DIR = os.path.join(os.path.dirname(__file__), 'output')
DOC_DIR = os.path.join(os.path.dirname(__file__), 'doc')


def load_dataset():
    df = pd.read_csv(os.path.join(OUTPUT_DIR, 'dataset.csv'))
    feature_cols = [c for c in df.columns if c not in ['set_num', 'theme']]
    X = df[feature_cols].values
    le = LabelEncoder()
    y = le.fit_transform(df['theme'])
    return X, y, feature_cols, le, df


def get_models_fast():
    """Models for experiments that run many iterations (exp1)."""
    return {
        'Random Forest': RandomForestClassifier(
            n_estimators=200, max_depth=None, class_weight='balanced',
            random_state=42, n_jobs=-1),
        'Hist Gradient Boosting': HistGradientBoostingClassifier(
            max_iter=200, max_depth=8, learning_rate=0.1, random_state=42),
    }


def get_models():
    return {
        'Random Forest': RandomForestClassifier(
            n_estimators=300, max_depth=None, class_weight='balanced',
            random_state=42, n_jobs=-1),
        'Hist Gradient Boosting': HistGradientBoostingClassifier(
            max_iter=300, max_depth=8, learning_rate=0.1, random_state=42),
        'MLP': MLPClassifier(
            hidden_layer_sizes=(256, 128, 64), activation='relu',
            max_iter=300, early_stopping=True, validation_fraction=0.15,
            random_state=42),
    }
```

```
def cv_evaluate(X, y, model_template, n_splits=5):
    """Run stratified CV and return mean accuracy and F1."""
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)
    accs, f1s = [], []
    for train_idx, test_idx in skf.split(X, y):
        scaler = StandardScaler()
        X_train = scaler.fit_transform(X[train_idx])
        X_test = scaler.transform(X[test_idx])
        model = clone(model_template)
        model.fit(X_train, y[train_idx])
        y_pred = model.predict(X_test)
        accs.append(accuracy_score(y[test_idx], y_pred))
        f1s.append(f1_score(y[test_idx], y_pred, average='macro', zero_division=0))
    return np.mean(accs), np.std(accs), np.mean(f1s), np.std(f1s)


# ============================================================
# Experiment 1: Training data amount
# ============================================================
def exp1_data_amount(X, y):
    print("\n=== Experiment 1: Training Data Amount ===")
    fractions = [0.2, 0.4, 0.6, 0.8, 1.0]
    models = get_models_fast()
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    results = []

    for model_name, model_template in models.items():
        for frac in fractions:
            accs = []
            for train_idx, test_idx in skf.split(X, y):
                X_train_full, y_train_full = X[train_idx], y[train_idx]
                X_test, y_test = X[test_idx], y[test_idx]

                # Subsample training data
                if frac < 1.0:
                    n_sub = max(1, int(len(X_train_full) * frac))
                    rng = np.random.RandomState(42)
                    idx = rng.choice(len(X_train_full), n_sub, replace=False)
                    X_train_sub, y_train_sub = X_train_full[idx], y_train_full[idx]
                else:
                    X_train_sub, y_train_sub = X_train_full, y_train_full

                scaler = StandardScaler()
                X_tr = scaler.fit_transform(X_train_sub)
                X_te = scaler.transform(X_test)
                model = clone(model_template)
                model.fit(X_tr, y_train_sub)
                accs.append(accuracy_score(y_test, model.predict(X_te)))

            mean_acc = np.mean(accs)
            std_acc = np.std(accs)
            results.append({'model': model_name, 'fraction': frac,
                            'accuracy': mean_acc, 'accuracy_std': std_acc})
            print(f"  {model_name}, frac={frac:.1f}: Acc={mean_acc:.4f}±{std_acc:.4f}")

    return results


# ============================================================
# Experiment 2: Class balance strategies
# ============================================================
def exp2_class_balance(X, y):
    print("\n=== Experiment 2: Class Balance Strategies ===")
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    results = []

    strategies = {
        'Original': {},
        'Balanced (class_weight)': {'class_weight': 'balanced'},
    }

    # RF and HGB with different class_weight settings
    for strategy_name, extra_params in strategies.items():
        for model_name_base, model_class, default_params in [
            ('Random Forest', RandomForestClassifier,
             dict(n_estimators=300, max_depth=None, random_state=42, n_jobs=-1)),
            ('Hist Gradient Boosting', HistGradientBoostingClassifier,
             dict(max_iter=300, max_depth=8, learning_rate=0.1, random_state=42)),
        ]:
            params = {**default_params}
            if 'class_weight' in extra_params and model_class == RandomForestClassifier:
                params['class_weight'] = extra_params['class_weight']
            elif 'class_weight' in extra_params and model_class == HistGradientBoostingClassifier:
                params['class_weight'] = extra_params['class_weight']

            model_template = model_class(**params)
            acc_m, acc_s, f1_m, f1_s = cv_evaluate(X, y, model_template)
            results.append({'model': model_name_base, 'strategy': strategy_name,
                            'accuracy': acc_m, 'accuracy_std': acc_s,
                            'f1': f1_m, 'f1_std': f1_s})
            print(f"  {model_name_base}, {strategy_name}: Acc={acc_m:.4f}, F1={f1_m:.4f}")

    # SMOTE
    print("  Running SMOTE experiments...")
```

```
    try:
        from imblearn.over_sampling import SMOTE
        smote = SMOTE(random_state=42)
        for model_name_base, model_template in [
            ('Random Forest', RandomForestClassifier(
                n_estimators=300, max_depth=None, random_state=42, n_jobs=-1)),
            ('Hist Gradient Boosting', HistGradientBoostingClassifier(
                max_iter=300, max_depth=8, learning_rate=0.1, random_state=42)),
        ]:
            accs, f1s = [], []
            for train_idx, test_idx in skf.split(X, y):
                scaler = StandardScaler()
                X_train = scaler.fit_transform(X[train_idx])
                X_test = scaler.transform(X[test_idx])
                X_res, y_res = smote.fit_resample(X_train, y[train_idx])
                model = clone(model_template)
                model.fit(X_res, y_res)
                y_pred = model.predict(X_test)
                accs.append(accuracy_score(y[test_idx], y_pred))
                f1s.append(f1_score(y[test_idx], y_pred, average='macro', zero_division=0))
            results.append({'model': model_name_base, 'strategy': 'SMOTE',
                            'accuracy': np.mean(accs), 'accuracy_std': np.std(accs),
                            'f1': np.mean(f1s), 'f1_std': np.std(f1s)})
            print(f"  {model_name_base}, SMOTE: Acc={np.mean(accs):.4f}, F1={np.mean(f1s):.4f}")
    except ImportError:
        print("  imbalanced-learn not installed, skipping SMOTE")

    return results


# ============================================================
# Experiment 3: PCA dimensionality reduction
# ============================================================
def exp3_pca(X, y):
    print("\n=== Experiment 3: PCA Dimensionality Reduction ===")
    n_components_list = [10, 30, 50, 100, 150]
    rf = RandomForestClassifier(
        n_estimators=300, max_depth=None, class_weight='balanced',
        random_state=42, n_jobs=-1)

    results = []

    # Full PCA for explained variance
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
    pca_full = PCA(random_state=42)
    pca_full.fit(X_scaled)
    explained_var = pca_full.explained_variance_ratio_.cumsum().tolist()

    # Baseline: no PCA
    acc_m, acc_s, f1_m, f1_s = cv_evaluate(X, y, rf)
    results.append({'n_components': X.shape[1], 'accuracy': acc_m,
                    'accuracy_std': acc_s, 'f1': f1_m, 'label': 'Full'})
    print(f"  Full ({X.shape[1]} dims): Acc={acc_m:.4f}")

    # PCA variants
    for n_comp in n_components_list:
        skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
        accs, f1s = [], []
        for train_idx, test_idx in skf.split(X, y):
            scaler = StandardScaler()
            X_train = scaler.fit_transform(X[train_idx])
            X_test = scaler.transform(X[test_idx])
            pca = PCA(n_components=n_comp, random_state=42)
            X_train_pca = pca.fit_transform(X_train)
            X_test_pca = pca.transform(X_test)
            model = clone(rf)
            model.fit(X_train_pca, y[train_idx])
            y_pred = model.predict(X_test_pca)
            accs.append(accuracy_score(y[test_idx], y_pred))
            f1s.append(f1_score(y[test_idx], y_pred, average='macro', zero_division=0))

        results.append({'n_components': n_comp, 'accuracy': np.mean(accs),
                        'accuracy_std': np.std(accs), 'f1': np.mean(f1s),
                        'label': str(n_comp)})
        print(f"  PCA({n_comp}): Acc={np.mean(accs):.4f}")

    return results, explained_var


# ============================================================
# Experiment 4: Feature ablation
# ============================================================
def exp4_feature_ablation(X, y, feature_cols):
    print("\n=== Experiment 4: Feature Ablation ===")
    rf = RandomForestClassifier(
        n_estimators=300, max_depth=None, class_weight='balanced',
        random_state=42, n_jobs=-1)

    # Define feature groups
    color_idx = [i for i, c in enumerate(feature_cols) if c.startswith('color_')]
    cat_idx = [i for i, c in enumerate(feature_cols) if c.startswith('cat_')]
    color_stat_idx = [i for i, c in enumerate(feature_cols)
                      if c in ['avg_r', 'avg_g', 'avg_b', 'std_r', 'std_g', 'std_b', 'prop_transparent']]
    meta_idx = [i for i, c in enumerate(feature_cols)
```

```
                if c in ['num_parts', 'year', 'num_unique_parts', 'num_unique_colors',
                         'num_unique_categories', 'num_minifigs', 'num_unique_minifigs']
                or c.startswith('mat_')]

    groups = {
        'Color features': color_idx + color_stat_idx,
        'Part category features': cat_idx,
        'Metadata features': meta_idx,
        'All features': list(range(len(feature_cols))),
    }

    results = []
    for group_name, idx in groups.items():
        X_sub = X[:, idx]
        acc_m, acc_s, f1_m, f1_s = cv_evaluate(X_sub, y, rf)
        results.append({'feature_group': group_name, 'n_features': len(idx),
                        'accuracy': acc_m, 'accuracy_std': acc_s,
                        'f1': f1_m, 'f1_std': f1_s})
        print(f"  {group_name} ({len(idx)} dims): Acc={acc_m:.4f}, F1={f1_m:.4f}")

    return results


# ============================================================
# Experiment 5: Hyperparameter sensitivity
# ============================================================
def exp5_hyperparameters(X, y):
    print("\n=== Experiment 5: Hyperparameter Sensitivity ===")
    results_estimators = []
    results_depth = []

    # Vary n_estimators
    for n_est in [10, 50, 100, 200, 500]:
        rf = RandomForestClassifier(
            n_estimators=n_est, max_depth=None, class_weight='balanced',
            random_state=42, n_jobs=-1)
        acc_m, acc_s, f1_m, f1_s = cv_evaluate(X, y, rf)
        results_estimators.append({'n_estimators': n_est, 'accuracy': acc_m,
                                   'accuracy_std': acc_s, 'f1': f1_m})
        print(f"  n_estimators={n_est}: Acc={acc_m:.4f}")

    # Vary max_depth
    for depth in [5, 10, 20, 50, None]:
        rf = RandomForestClassifier(
            n_estimators=300, max_depth=depth, class_weight='balanced',
            random_state=42, n_jobs=-1)
        acc_m, acc_s, f1_m, f1_s = cv_evaluate(X, y, rf)
        depth_label = str(depth) if depth else 'None'
        results_depth.append({'max_depth': depth_label, 'accuracy': acc_m,
                              'accuracy_std': acc_s, 'f1': f1_m})
        print(f"  max_depth={depth_label}: Acc={acc_m:.4f}")

    return results_estimators, results_depth


def run():
    X, y, feature_cols, le, df = load_dataset()

    all_results = {}
    all_results['exp1'] = exp1_data_amount(X, y)
    all_results['exp2'] = exp2_class_balance(X, y)
    all_results['exp3_results'], all_results['exp3_explained_var'] = exp3_pca(X, y)
    all_results['exp4'] = exp4_feature_ablation(X, y, feature_cols)
    all_results['exp5_estimators'], all_results['exp5_depth'] = exp5_hyperparameters(X, y)

    # Save all experiment results
    results_path = os.path.join(OUTPUT_DIR, 'experiment_results.json')
    with open(results_path, 'w') as f:
        json.dump(all_results, f, indent=2)

    # Write experiment log
    os.makedirs(DOC_DIR, exist_ok=True)
    log_path = os.path.join(DOC_DIR, 'experiment_log.md')
    with open(log_path, 'w') as f:
        f.write("# Experiment Log\n\n")

        f.write("## Experiment 1: Training Data Amount\n\n")
        f.write("| Model | 20% | 40% | 60% | 80% | 100% |\n")
        f.write("|-------|-----|-----|-----|-----|------|\n")
        exp1_df = pd.DataFrame(all_results['exp1'])
        for model in exp1_df['model'].unique():
            row = exp1_df[exp1_df['model'] == model]
            vals = [f"{row[row['fraction']==fr]['accuracy'].values[0]:.4f}" for fr in [0.2,0.4,0.6,0.8,1.0]]
            f.write(f"| {model} | {' | '.join(vals)} |\n")

        f.write("\n## Experiment 2: Class Balance Strategies\n\n")
        f.write("| Model | Strategy | Accuracy | F1 (Macro) |\n")
        f.write("|-------|----------|----------|------------|\n")
        for r in all_results['exp2']:
            f.write(f"| {r['model']} | {r['strategy']} | {r['accuracy']:.4f} | {r['f1']:.4f} |\n")

        f.write("\n## Experiment 3: PCA Dimensionality Reduction\n\n")
        f.write("| Components | Accuracy |\n")
        f.write("|------------|----------|\n")
        for r in all_results['exp3_results']:
```

```
            f.write(f"| {r['label']} | {r['accuracy']:.4f} |\n")

        f.write("\n## Experiment 4: Feature Ablation\n\n")
        f.write("| Feature Group | # Features | Accuracy | F1 (Macro) |\n")
        f.write("|---------------|------------|----------|------------|\n")
        for r in all_results['exp4']:
            f.write(f"| {r['feature_group']} | {r['n_features']} | {r['accuracy']:.4f} | {r['f1']:.4f} |\n")

        f.write("\n## Experiment 5: Hyperparameter Sensitivity\n\n")
        f.write("### n_estimators\n\n")
        f.write("| n_estimators | Accuracy |\n")
        f.write("|--------------|----------|\n")
        for r in all_results['exp5_estimators']:
            f.write(f"| {r['n_estimators']} | {r['accuracy']:.4f} |\n")

        f.write("\n### max_depth\n\n")
        f.write("| max_depth | Accuracy |\n")
        f.write("|-----------|----------|\n")
        for r in all_results['exp5_depth']:
            f.write(f"| {r['max_depth']} | {r['accuracy']:.4f} |\n")

    print(f"\nExperiment results saved to {results_path}")
    print(f"Experiment log saved to {log_path}")
    return all_results


if __name__ == '__main__':
    run()
```

visualize.py

Generate report figures

```
"""
Visualization: Generate all figures for the report.

Reads results from output/ and produces PNG figures in figures/.
"""

import os
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.manifold import TSNE

# Global style
plt.rcParams.update({
    'font.size': 12,
    'figure.dpi': 150,
    'savefig.bbox': 'tight',
    'savefig.pad_inches': 0.2,
})

OUTPUT_DIR = os.path.join(os.path.dirname(__file__), 'output')
FIG_DIR = os.path.join(os.path.dirname(__file__), 'figures')


def load_results():
    with open(os.path.join(OUTPUT_DIR, 'baseline_results.json')) as f:
        baseline = json.load(f)
    with open(os.path.join(OUTPUT_DIR, 'experiment_results.json')) as f:
        experiments = json.load(f)
    with open(os.path.join(OUTPUT_DIR, 'class_names.json')) as f:
        class_names = json.load(f)
    with open(os.path.join(OUTPUT_DIR, 'feature_names.json')) as f:
        feature_names = json.load(f)
    with open(os.path.join(OUTPUT_DIR, 'feature_importance.json')) as f:
        importance = json.load(f)
    return baseline, experiments, class_names, feature_names, importance


def fig1_class_distribution():
    """Bar chart of samples per theme."""
    df = pd.read_csv(os.path.join(OUTPUT_DIR, 'dataset.csv'))
    counts = df['theme'].value_counts().sort_values()

    fig, ax = plt.subplots(figsize=(8, 5))
    colors = sns.color_palette('Set2', len(counts))
    bars = ax.barh(counts.index, counts.values, color=colors)
    ax.set_xlabel('Number of Sets')
    ax.set_title('Dataset: Samples per LEGO Theme')
    for bar, val in zip(bars, counts.values):
        ax.text(bar.get_width() + 10, bar.get_y() + bar.get_height()/2,
                str(val), va='center', fontsize=10)
    plt.savefig(os.path.join(FIG_DIR, 'class_distribution.png'))
    plt.close()
    print("  [1/11] class_distribution.png")
```

```
def fig2_tsne():
    """t-SNE visualization of feature space."""
    df = pd.read_csv(os.path.join(OUTPUT_DIR, 'dataset.csv'))
    feature_cols = [c for c in df.columns if c not in ['set_num', 'theme']]
    X = df[feature_cols].values
    y = df['theme'].values

    # Subsample for speed
    rng = np.random.RandomState(42)
    n = min(3000, len(X))
    idx = rng.choice(len(X), n, replace=False)
    X_sub, y_sub = X[idx], y[idx]

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X_sub)
    tsne = TSNE(n_components=2, perplexity=30, random_state=42, max_iter=1000)
    X_2d = tsne.fit_transform(X_scaled)

    themes = sorted(set(y_sub))
    fig, ax = plt.subplots(figsize=(10, 8))
    palette = sns.color_palette('tab10', len(themes))
    for i, theme in enumerate(themes):
        mask = y_sub == theme
        ax.scatter(X_2d[mask, 0], X_2d[mask, 1], c=[palette[i]], label=theme,
                   alpha=0.6, s=15, edgecolors='none')
    ax.legend(fontsize=9, markerscale=2, loc='best')
    ax.set_title('t-SNE Visualization of LEGO Set Features')
    ax.set_xlabel('t-SNE 1')
    ax.set_ylabel('t-SNE 2')
    plt.savefig(os.path.join(FIG_DIR, 'tsne_visualization.png'))
    plt.close()
    print("  [2/11] tsne_visualization.png")


def fig3_confusion_matrices(baseline, class_names):
    """Confusion matrix heatmaps for each model."""
    for i, (model_name, res) in enumerate(baseline.items()):
        cm = np.array(res['confusion_matrix'])
        # Normalize by row (true labels)
        cm_norm = cm / cm.sum(axis=1, keepdims=True)

        fig, ax = plt.subplots(figsize=(8, 7))
        sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Blues',
                    xticklabels=class_names, yticklabels=class_names, ax=ax,
                    vmin=0, vmax=1, annot_kws={'size': 9})
        ax.set_xlabel('Predicted')
        ax.set_ylabel('True')
        ax.set_title(f'Confusion Matrix: {model_name}')
        plt.xticks(rotation=45, ha='right')
        plt.yticks(rotation=0)
        fname = f'confusion_matrix_{model_name.lower().replace(" ", "_")}.png'
        plt.savefig(os.path.join(FIG_DIR, fname))
        plt.close()
        print(f"  [{3+i}/11] {fname}")


def fig4_learning_curves(experiments):
    """Accuracy vs training data fraction for each model."""
    exp1 = pd.DataFrame(experiments['exp1'])

    fig, ax = plt.subplots(figsize=(8, 5))
    markers = ['o', 's', '^']
    for i, model in enumerate(exp1['model'].unique()):
        data = exp1[exp1['model'] == model]
        ax.errorbar(data['fraction'], data['accuracy'], yerr=data['accuracy_std'],
                    marker=markers[i], label=model, capsize=3, linewidth=2)
    ax.set_xlabel('Training Data Fraction')
    ax.set_ylabel('Accuracy')
    ax.set_title('Effect of Training Data Amount')
    ax.legend()
    ax.set_xticks([0.2, 0.4, 0.6, 0.8, 1.0])
    ax.grid(True, alpha=0.3)
    plt.savefig(os.path.join(FIG_DIR, 'learning_curves.png'))
    plt.close()
    print("  [6/11] learning_curves.png")


def fig5_pca_explained_variance(experiments):
    """Cumulative explained variance vs number of components."""
    explained = experiments['exp3_explained_var']

    fig, ax = plt.subplots(figsize=(8, 5))
    ax.plot(range(1, len(explained)+1), explained, linewidth=2, color='steelblue')
    # Mark key thresholds
    for thresh in [0.90, 0.95, 0.99]:
        n = next((i+1 for i, v in enumerate(explained) if v >= thresh), len(explained))
        ax.axhline(y=thresh, color='gray', linestyle='--', alpha=0.5)
        ax.axvline(x=n, color='gray', linestyle='--', alpha=0.5)
        ax.annotate(f'{thresh*100:.0f}% at n={n}', xy=(n, thresh),
                    fontsize=9, ha='left', va='bottom')
    ax.set_xlabel('Number of Components')
    ax.set_ylabel('Cumulative Explained Variance')
    ax.set_title('PCA: Explained Variance')
```

```
    ax.grid(True, alpha=0.3)
    plt.savefig(os.path.join(FIG_DIR, 'pca_explained_variance.png'))
    plt.close()
    print("  [7/11] pca_explained_variance.png")


def fig6_pca_accuracy(experiments):
    """Accuracy vs number of PCA components."""
    pca_results = experiments['exp3_results']

    fig, ax = plt.subplots(figsize=(8, 5))
    labels = [r['label'] for r in pca_results]
    accs = [r['accuracy'] for r in pca_results]
    x_pos = range(len(labels))
    bars = ax.bar(x_pos, accs, color='steelblue', alpha=0.8)
    ax.set_xticks(x_pos)
    ax.set_xticklabels(labels)
    ax.set_xlabel('Number of PCA Components')
    ax.set_ylabel('Accuracy')
    ax.set_title('PCA: Effect on Classification Accuracy (Random Forest)')
    ax.set_ylim(min(accs) - 0.02, max(accs) + 0.01)
    for bar, acc in zip(bars, accs):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.002,
                f'{acc:.4f}', ha='center', va='bottom', fontsize=10)
    ax.grid(True, alpha=0.3, axis='y')
    plt.savefig(os.path.join(FIG_DIR, 'pca_accuracy.png'))
    plt.close()
    print("  [8/11] pca_accuracy.png")


def fig7_class_balance(experiments):
    """Grouped bar chart: F1 per model per balancing strategy."""
    exp2 = pd.DataFrame(experiments['exp2'])

    fig, ax = plt.subplots(figsize=(9, 5))
    models = exp2['model'].unique()
    strategies = exp2['strategy'].unique()
    x = np.arange(len(models))
    width = 0.25

    for i, strategy in enumerate(strategies):
        data = exp2[exp2['strategy'] == strategy]
        f1_vals = [data[data['model'] == m]['f1'].values[0] for m in models]
        bars = ax.bar(x + i * width, f1_vals, width, label=strategy, alpha=0.85)

    ax.set_xlabel('Model')
    ax.set_ylabel('Macro F1 Score')
    ax.set_title('Effect of Class Balance Strategy')
    ax.set_xticks(x + width)
    ax.set_xticklabels(models, rotation=15, ha='right')
    ax.legend()
    ax.grid(True, alpha=0.3, axis='y')
    plt.savefig(os.path.join(FIG_DIR, 'class_balance_comparison.png'))
    plt.close()
    print("  [9/11] class_balance_comparison.png")


def fig8_feature_importance(importance, feature_names):
    """Top 30 features by Random Forest importance."""
    sorted_imp = sorted(importance.items(), key=lambda x: x[1], reverse=True)[:30]
    names = [x[0] for x in sorted_imp][::-1]
    values = [x[1] for x in sorted_imp][::-1]

    # Map feature names to readable labels
    def readable_name(name):
        if name.startswith('cat_'):
            # Load part categories
            try:
                cats = pd.read_csv(os.path.join(os.path.dirname(__file__), 'data', 'part_categories.csv'))
                cat_id = int(name.replace('cat_', ''))
                match = cats[cats['id'] == cat_id]
                if not match.empty:
                    return f"Cat: {match.iloc[0]['name']}"
            except:
                pass
        elif name.startswith('color_'):
            try:
                colors = pd.read_csv(os.path.join(os.path.dirname(__file__), 'data', 'colors.csv'))
                color_id = int(name.replace('color_', ''))
                match = colors[colors['id'] == color_id]
                if not match.empty:
                    return f"Color: {match.iloc[0]['name']}"
            except:
                pass
        return name

    readable = [readable_name(n) for n in names]

    fig, ax = plt.subplots(figsize=(9, 8))
    ax.barh(range(len(readable)), values, color='steelblue', alpha=0.8)
    ax.set_yticks(range(len(readable)))
    ax.set_yticklabels(readable, fontsize=9)
    ax.set_xlabel('Feature Importance')
    ax.set_title('Top 30 Features (Random Forest)')
    ax.grid(True, alpha=0.3, axis='x')
```

```
    plt.savefig(os.path.join(FIG_DIR, 'feature_importance.png'))
    plt.close()
    print("  [10/11] feature_importance.png")


def fig9_feature_ablation(experiments):
    """Bar chart: accuracy per feature subset."""
    exp4 = experiments['exp4']

    fig, ax = plt.subplots(figsize=(8, 5))
    groups = [r['feature_group'] for r in exp4]
    accs = [r['accuracy'] for r in exp4]
    f1s = [r['f1'] for r in exp4]
    x = np.arange(len(groups))
    width = 0.35

    bars1 = ax.bar(x - width/2, accs, width, label='Accuracy', alpha=0.85)
    bars2 = ax.bar(x + width/2, f1s, width, label='Macro F1', alpha=0.85)

    ax.set_ylabel('Score')
    ax.set_title('Feature Ablation Study')
    ax.set_xticks(x)
    ax.set_xticklabels(groups, rotation=20, ha='right', fontsize=10)
    ax.legend()
    ax.set_ylim(min(min(accs), min(f1s)) - 0.05, 1.0)
    ax.grid(True, alpha=0.3, axis='y')

    for bar, val in zip(bars1, accs):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
                f'{val:.3f}', ha='center', va='bottom', fontsize=9)
    for bar, val in zip(bars2, f1s):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
                f'{val:.3f}', ha='center', va='bottom', fontsize=9)

    plt.savefig(os.path.join(FIG_DIR, 'feature_ablation.png'))
    plt.close()
    print("  [11/11] feature_ablation.png")


def fig10_hyperparams(experiments):
    """Hyperparameter sensitivity plots."""
    # n_estimators
    est_data = experiments['exp5_estimators']
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    ax = axes[0]
    x = [r['n_estimators'] for r in est_data]
    y = [r['accuracy'] for r in est_data]
    ax.plot(x, y, 'o-', linewidth=2, color='steelblue')
    ax.set_xlabel('n_estimators')
    ax.set_ylabel('Accuracy')
    ax.set_title('RF: Effect of n_estimators')
    ax.grid(True, alpha=0.3)

    # max_depth
    depth_data = experiments['exp5_depth']
    ax = axes[1]
    labels = [r['max_depth'] for r in depth_data]
    y = [r['accuracy'] for r in depth_data]
    ax.plot(range(len(labels)), y, 'o-', linewidth=2, color='coral')
    ax.set_xticks(range(len(labels)))
    ax.set_xticklabels(labels)
    ax.set_xlabel('max_depth')
    ax.set_ylabel('Accuracy')
    ax.set_title('RF: Effect of max_depth')
    ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig(os.path.join(FIG_DIR, 'hyperparameter_sensitivity.png'))
    plt.close()
    print("  [bonus] hyperparameter_sensitivity.png")


def run():
    print("=== Generating Figures ===")
    os.makedirs(FIG_DIR, exist_ok=True)

    baseline, experiments, class_names, feature_names, importance = load_results()

    fig1_class_distribution()
    fig2_tsne()
    fig3_confusion_matrices(baseline, class_names)
    fig4_learning_curves(experiments)
    fig5_pca_explained_variance(experiments)
    fig6_pca_accuracy(experiments)
    fig7_class_balance(experiments)
    fig8_feature_importance(importance, feature_names)
    fig9_feature_ablation(experiments)
    fig10_hyperparams(experiments)

    print(f"\nAll figures saved to {FIG_DIR}/")


if __name__ == '__main__':
    run()
```